Amit Kurmoli

SDET / QA Automation Engineer

Bangalore, India • amitkurmoli79@gmail.com • LinkedIn • GitHub

SUMMARY

SDET / QA Automation Engineer with 5+ years designing and delivering test automation across web, desktop, and mobile applications. Strong in Python with practical depth in Playwright, pytest, REST and GraphQL API testing, performance testing, and CI/CD pipeline integration - building reusable, CI-integrated frameworks from scratch and owning QA on production projects. Recent M.S. in Information Science (Machine Learning emphasis), applying data-pipeline and dataset experience to how test data is structured, validated, and reasoned about in quality engineering.

TECHNICAL SKILLS

Test Automation: Python, pytest, Playwright, Selenium WebDriver, Appium, Winium, Java; Page Object Model, fixture design, parallel execution, flaky-test handling, framework design from scratch

API Testing: REST and GraphQL; httpx, requests, REST Assured; contract testing (Pact), schema-based fuzzing (Schemathesis), authentication flows, error-as-data vs transport-error handling

Performance & Load: k6, JMeter; load and stress testing, SLO thresholds, fault injection (Toxiproxy), observability (Prometheus, Grafana)

CI / CD & Infrastructure: GitHub Actions, Jenkins, Docker, Docker Compose, pipeline design and quality gates; AWS familiarity (EC2, S3, IAM)

Backend, Data & Reporting: FastAPI, SQL, SQLite, Pydantic; database validation in test suites; Allure and HTML reporting; Git, agile delivery

Cross-Platform Testing: iOS and Android device matrix, React Native context, cross-browser (Chrome, Firefox, Safari), Windows desktop UI automation, accessibility testing (WCAG)

Machine Learning (M.S. coursework): Python, TensorFlow, Transformer architectures, neural networks, computer vision, data pipelines, MediaPipe

PROFESSIONAL EXPERIENCE

Quality Engineer — Pepper Square Bangalore, India | 2022 – 2024

Led quality operations and served as the primary point of contact in a service-based consultancy

- Established the QA function from scratch — defined testing standards, processes, and a reusable Java/Selenium automation framework covering websites, web applications, and mobile applications.
- Right-sized the framework deliberately: integrated with CI for fast commit feedback, with unit and end-to-end suites cleanly separated and engineering effort balanced against meaningful coverage rather than over-investing in tooling.
- Led QA on the ground-up revamp of a React Native mobile application (iOS and Android) for a US-based fitness organization — manual functional testing, Appium UI automation, REST Assured API testing, and SQL-based database validation across the UI, service, and data layers.
- Bridged design and development by translating Figma designs into structured requirements and acceptance criteria — reducing ambiguity before code was written.
- Led accessibility testing for a global pharmaceutical company's website revamp — validated WCAG compliance across keyboard navigation, colour contrast and high-contrast mode, 200% zoom scaling, captions and transcripts, form-field labels, and skip-navigation behaviour.
- Ran cross-browser (Chrome/Firefox/Safari) and cross-platform device-matrix testing across iOS and Android phones and tablets, including the mobile web revamp of a major national restaurant brand.

Member Technical Staff — Quality, Sharp Japan-based product company | 2019 – 2022

QA across Sharp's document-solutions products — web, Windows desktop, and mobile

- COCORO OFFICE Shared Folder: manual functional testing of file-sharing workflows — upload, download, permission enforcement, secure share links — plus Java + Selenium WebDriver web-portal automation inside an established enterprise test framework.

- Sharpdesk Mobile: manual functional testing of print, scan, and file-sharing workflows across iOS and Android, and Java + Appium UI automation covering variation in devices, screen sizes, and OS versions.
- Sharpdesk Desktop: manual functional testing across document, scanning, OCR, and annotation workflows, and Java + Winium UI regression automation (Selenium-style WebDriver for Windows desktop applications).

INDEPENDENT & RESEARCH PROJECTS

Anton — Test Automation Framework Independent project | 2025 – present

Four-engine test framework with one shared interface

- Designed and built a four-engine framework — API, UI, performance, and chaos — against Vendure, an open-source GraphQL e-commerce platform. Every engine implements the same prepare → execute → collect artifacts interface, so orchestration, fixtures, and reporting are shared across all test types.
- API: pytest with a GraphQL client over requests. UI: Playwright with Page Object Model. Performance: k6 with SLO thresholds. Chaos: fault injection via real network-level tooling.
- Pytest plugin writes every run to SQLite; a FastAPI + React dashboard reads from it; GitHub Actions runs all 44 tests on every push with downloadable HTML reports.
- Stack: Python, pytest, Playwright, GraphQL, k6, FastAPI, React, SQLite, Docker Compose, GitHub Actions.
- Actively developed — repository: github.com/Ak79p/anton

API Test Framework — Quality-Gated API Automation Independent project | 2026

Layered test pyramid with a staged smoke → nightly → release CI strategy

- Designed a three-tier CI/CD quality-gate strategy in GitHub Actions — a fast smoke pipeline on every push for developer feedback, a nightly pipeline running full regression plus contract, k6 performance, and coverage, and a release pipeline that blocks version tagging until every gate passes.
- Structured the suite as a layered test pyramid — unit (mocked), integration (CRUD against a live FastAPI service), contract (OpenAPI validation via Schemathesis), and end-to-end journeys — with pytest markers for selective, parallel-safe runs.
- Built automatic failure diagnostics capturing request method, URL, payload, and response on any failed test, surfaced through JUnit XML for CI and Allure for human investigation.
- Stack: Python, pytest, FastAPI, Schemathesis, k6, Ruff, Allure, GitHub Actions.
  Repository: github.com/Ak79p/api-test-framework

ASL-TRM — Transformer Model for ASL Gesture Recognition M.S. Capstone | 2025 – 2026

Three-person team — owned the data pipeline; contributed to model development

- Built the end-to-end data pipeline for a lightweight Transformer-based ASL gesture recognition system: dataset preparation (WLASL, Microsoft ASL-Citizen), MediaPipe Holistic feature extraction, and data ingestion for training and evaluation.
- Assisted in developing TRM-Micro, adapting recursive-reasoning principles from recent Transformer research to the spatiotemporal domain of sign language.
- TRM-Micro v3 matched or exceeded the ST-GCN baseline on ASL-Citizen (Top-1: 61.46% vs 59.52%) at roughly one-fifth the parameter count (~622K vs ~3.1M).
- Stack: Python, TensorFlow, Transformer architectures, MediaPipe Holistic, Streamlit.
  Repository: github.com/Ak79p/asl_trm

EDUCATION

M.S. Information Science, University of Arizona 2024 – 2026

Machine Learning emphasis

M.Tech Power Electronics 2016 – 2018

B.E. Electronics & Electrical Engineering 2012 – 2016

PORTFOLIO

Portfolio: Ak79p.github.io